COM 311 (OPERATIZING SYSTEM I)

1.0 TYPES OF OPERATING SYSTEMS

1.1 What is an Operating System?

An Operating System (OS) is a collection of system programs that controls and coordinates the overall operation of a computer system. Operating system is the most basic program within the computer system. These programs act as an interface between the hardware, application programs, files and user. In other words, we can say that an operating system is a communication link between the user and computer and helps the user to run the application programs. The operating system manages the computer resources such as CPU, memory and I/O Devices. The smaller, part the nucleus or kernel of the operating system is memory resident and hence called the resident part of the operating system. The other larger part of the operating system is maintained in the secondary storage i.e. this part is transient which means it swaps in and out where and when needed by the user.

Operation System Definition

An integrated system of programs which supervises the operations of the CPU, controls the input/output and storage functions of the computer system and provides various support services as the computer executes the application programs of users.

1.2 Importance

Following are the points that justify the need and importance of an operating system

1. Operating system behaves as a resource manager. It utilizes the computer in a cost effective manner. It keeps account of different jobs and the where about of their results and locations in the memory.
2. It schedules jobs according to their priority passing control from one program to the next. The overall function of job control is especially important where there are several users (a multi user environment).
3. Operating system makes a communication link between user and the system and helps the user to run application programs properly and get the required output.
4. Operating system has the ability to fetch the programs in the memory when required and not all the operating system to be loaded in the memory at the same time. Thus giving the user the space to work in the required package more conveniently and easily.
5. Operating system helps the user in file management, making of directions, and saving files in them, is a very good feature provided by the operating system to organize data according to the needs of the user.
6. Multiprogramming is a very important feature of operating system. It schedules and controls the running of several programs at once.
7. It provides program editors that help the user to modify and update the program lines.
8. Debugging aids provided by the operating system help the user to detect and rename errors in programs
9. Disk maintenance ability of operating system checks the validity of data stored on diskettes and perhaps make corrections to erroneous data.

1.3 classifications of operating systems

BATCH OPERATING SYSTEM:

"The operating system is termed as "batch operating" because the input data (job) are collected into batches or sets of records with similar needs and each batch is processed as a unit(group). The output is another batch that can be reused for computation."

- In this OS the user did not interact directly with the computer systems.
- The user prepared a job (input data) using off-line device like punch cards, and submitted it to the computer operator.
- After some time the output appeared. The output consisted of the result of the program, as well as a dump of the final memory and register contents for debugging.
- To speed up processing, operators batched together jobs with similar needs and ran them through the computer as a group. Thus, the programmers would leave their programs with the operator.
- The output from each job would be sent back to the appropriate programmer. In this execution environment, the CPU is often idle, because the speeds of the mechanical I/O devices are intrinsically slower than are those of electronic devices.
- Operating System's major task was to transfer control automatically from one job to the next. The operating system was always resident in memory.
- In batch environment process is created in response to the submission of a job.

Advantages:

- Batch processing takes much of the work of the operator to the computer.
- Increased performance as a new job gets started as soon as the previous job finished without any manual intervention.

Disadvantages:- Difficult to debug program.

- A job could enter an infinite loop.
- Due to the lack of protection scheme, one batch job can affect pending jobs.

REAL TIME OPERATING SYSTEM

The **real-time operating system** used for a real-time application means for those applications where data processing should be done in the fixed and small quantum of time. It is different from general purpose computer where time concept is not considered as much crucial as in Real-Time Operating System. RTOS is a time-sharing system based on clock interrupts. Interrupt Service Routine (ISR) serve the interrupt, raised by the system. RTOS used Priority to execute the process. When a high priority process enters in system low priority process preempted to serve higher priority process. Real-time operating system synchronized the process. So that they can communicate with each other. Resources can be used efficiently without wastage of time.

RTOS are controlling traffic signal; Nuclear reactors Control scientific experiments, medical imaging systems, industrial system, fuel injection system, home appliance are some application of Real Time operating system

Real time Operating Systems are very fast and quick respondent systems. These systems are used in an environment where a large number of events (generally external) must be accepted and processed in a short time. Real time processing requires quick transaction and characterized by supplying immediate response. For example, a measurement from a petroleum refinery indicating that temperature is getting too high and might demand for immediate attention to avoid an explosion.

In real time operating system there is a little swapping of programs between primary and secondary memory. Most of the time, processes remain in primary memory in order to provide quick response, therefore, memory management in real time system is less demanding compared to other systems.

Types of Real-Time Operating System

1) Soft Real-Time Operating System : A process might not be executed in given deadline. It can be crossed it then executed next, without harming the system. Example are a digital camera, mobile phones, etc.

2) Hard Real-Time Operating System : A process should be executed in given deadline. The deadline should not be crossed. Preemption time for Hard Real-Time Operating System is almost less than few microseconds.

Examples are Airbag control in cars, anti-lock brake, engine control system, etc.

TIME SHARING OPERATING SYSTEM:

As the name itself suggests, in a time-sharing system or multi-tasking system, multiple jobs can be executed on a system at the same time by sharing the CPU **time** among them. It is considered to be a **logical** extension of multiprogramming because both does simultaneous execution but differ in their prime objectives. The main objective of time-sharing systems is to **minimize response time** but not maximizing the processor use (which is the objective of multiprogramming systems).

- ***Advantages:***
  1. They minimize the **response time** i.e; greater the response time, lesser is the efficiency of the system.
  2. They reduce CPU **idle** time.
- ***Disadvantages:***
  1. Security and **Synchronization** must be given a greater importance.
  2. Data communication must be enabled.
- **Examples of Time-Sharing OSs are:** Multics, Unix etc.

NETWORK OPERATING SYSTEM:

Network Operating System(NOS) is similar to distributed systems **but** they differ in the way they access resources. An NOS need **special functions/protocols** to facilitate connectivity & communication among the systems. **NOS** employs a **client-server model** where as a **DOS** employs a **master-slave model.** In **NOS**, to process the data, it has to be transferred to the server.

- ***Types:***
  1. Peer-to-Peer model
  2. Client-Server model
- ***Objectives:***
  1. Providing Security.
  2. Facilitating user management i.e; logon & logoff.
  3. Enabling remote access to servers.
- ***Advantages:***
  1. Stabilized Servers.
  2. Provides file, print, web & back-up services.
  3. Authorized access & automatic hardware detection.
- ***Disadvantages:***
  1. Expensive as they need to run servers continuously.
  2. Need for regular maintenance & updates.
  3. Depends on the centre location (server) even for small operations.
- **Examples of Network Operating System are:** Microsoft Windows Server 2003, Microsoft Windows Server 2008, UNIX, Linux, Mac OS X, Novell NetWare, and BSD etc.

2.0 THE STRUCTURE, FUNCTIONS AND PHILOSOPHY ON OPERATING SYSTEMS

2.1 Operating System Resource Management

Resource management is the dynamic allocation and de-allocation by an operating system of processor cores, memory pages, and various types of bandwidth to computations that compete for those resources. The objective is to allocate resources so as to optimize responsiveness subject to the finite resources available.

Following are some of important functions of an operating System.

- Memory Management
- Processor Management
- Device Management
- File Management
- Security
- Control over system performance
- Job accounting
- Error detecting aids
- Coordination between other software and users

2.1.1 OS function in relation to memory management

Memory management refers to management of Primary Memory or Main Memory. Main memory is a large array of words or bytes where each word or byte has its own address.

Main memory provides a fast storage that can be access directly by the CPU. So for a program to be executed, it must in the main memory. Operating System does the following activities for memory management.

- Keeps tracks of primary memory i.e. what part of it are in use by whom, what part are not in use.
- In multiprogramming, OS decides which process will get memory when and how much.
- Allocates the memory when the process requests it to do so.
- De-allocates the memory when the process no longer needs it or has been terminated.

2.1.2 OS function in relation to interrupt handling

An interrupt is an input signal to the processor indicating an event that needs immediate attention. An interrupt signal alerts the processor and serves as a request for the processor to interrupt the currently executing code, so that the event can be processed in a timely manner. If the request is accepted, the processor responds by suspending its current activities, saving its state, and executing a function called an interrupt handler (or an interrupt service routine, ISR) to deal with the event. This interruption is temporary, and, unless the interrupt indicates a fatal error, the processor resumes normal activities after the interrupt handler finishes.

Interrupts are commonly used by hardware devices to indicate electronic or physical state changes that require attention. Interrupts are also commonly used to implement computer multitasking, especially in real-time computing. Systems that use interrupts in these ways are said to be interrupt-driven.

2.1.3 OS function in relation to Communication management:

Coordination and assignment of compilers, interpreters, and another software resource of the various users of the computer systems.

2.2 Characteristics and Features of Operating Systems

2.2.1 Operating System Characteristics

The Operating systems are different according to the three primary characteristics which are licensing, software compatibility, and complexity.

- **Licensing:** There are basically three kinds of Operating systems. One is Open Source OS, another is Free OS and the third is Commercial OS.

Linux is an ***open source*** operating system which means that anyone can download and modify it for example Ubuntu etc.

Commercial operating systems are privately owned by companies that charge money for them. Examples include Microsoft Windows and Apple mac OS. These require to pay for the right (or *license*) to use their Operating systems.

- **Software Compatibility:** The developers make the software which may be compatible or incompatible in different versions within the same operating system's type but they can't be compatible with the other OS types. Every OS types have their own software compatibility.
- **Complexity:** Operating systems come in basically two editions one is 32-bit and other is 64-bit editions. The 64-bit edition of an operating system best utilizes random access memory (RAM). A computer with a 64-bit CPU can run either a 32-bit or a 64-bit OS, but a computer with a 32-bit CPU can run only a 32-bit OS.

2.2.2 Features of Operating System

Here is a list commonly found important features of an Operating System:

- Protected and supervisor mode

- Allows disk access and file systems Device drivers Networking Security
- Program Execution
- Memory management Virtual Memory Multitasking
- Handling I/O operations
- Manipulation of the file system
- Error Detection and handling
- Resource allocation
- Information and Resource Protection

3.0 INTER PROCESS COMMUNICATION (IPC)

Processes executing concurrently in the operating system may be either independent processes or cooperating processes. A process is **independent** if it cannot affect or be affected by the other processes executing in the system. Any process that does not share data with any other process is independent. A process is **cooperating** if **it** can affect or be affected by the other processes executing in the system.

There are several reasons for providing an environment that allows process cooperation:

• **Information sharing.** Since several users may be interested in the same piece of information (for instance, a shared file), we must provide an environment to allow concurrent access to such information.

• **Computation speedup.** If we want a particular task to run faster, we must break it into subtasks, each of which will be executing in parallel with the others. Notice that such a speedup can be achieved only if the computer has multiple processing elements (such as CPUs or I/O channels).

• **Modularity.** We may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads.

• **Convenience.** Even an individual user may work on many tasks at the same time. For instance, a user may be editing, printing, and compiling in parallel.

Cooperating processes require an **inter process communication (IPC)** mechanism that will allow them to exchange data and information.

There are two fundamental models of inter process communication:

(1) shared memory and (2) **message passing.** In the shared-memory model, a region of memory that is shared by cooperating processes is established. Processes can then exchange information by reading and writing data to the shared region. In the message passing model, communication takes place by means of messages exchanged between the cooperating processes.

Message passing is useful for exchanging smaller amounts of data, because no conflicts need be avoided. Message passing is also easier to implement than is shared memory for inter computer communication. Shared memory allows maximum speed and convenience of communication, as it can be done at memory speeds when within a computer.

Shared memory is faster than message passing, as message-passing systems are typically implemented using system calls and thus require the more time consuming task of kernel intervention.

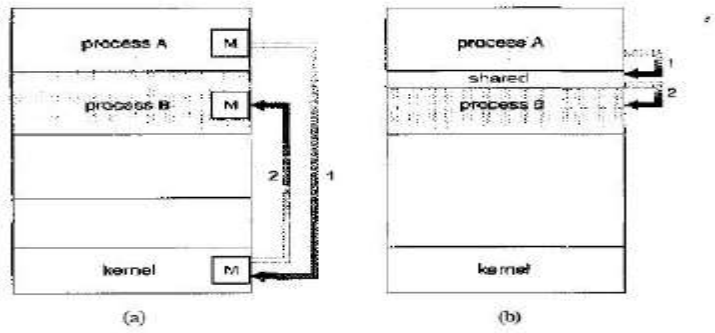


Figure 3.13 Communications models. (a) Message passing. (b) Shared memory.

Shared-Memory Systems: Inter process communication using shared memory requires communicating processes to establish a region of shared memory. Typically, a shared-memory region resides in the address space of the process creating the shared-memory segment. Other processes that wish to communicate using this shared-memory segment must attach it to their address space. The operating system tries to prevent one process from accessing another process's memory. Shared memory requires that two or more processes agree to remove this restriction. They can then exchange information by reading and writing data in the shared areas. The form of the data and the location are determined by these processes and are not under the operating system's control. The processes are also responsible for ensuring that they are not writing to the same location simultaneously.

Message-Passing Systems: The scheme requires that these processes share a region of memory and that the code for accessing and manipulating the shared memory be written explicitly by the application programmer. Another way to

achieve the same effect is for the operating system to provide the means for cooperating processes to communicate with each other via a message-passing facility.
Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space and is particularly useful in a distributed environment, where the communicating processes may reside on different computers connected by a network. A message-passing facility provides at least two operations: send(message) and receive(message).
Messages sent by a process can be of either fixed or variable size. If only fixed-sized messages can be sent, the system-level implementation is straightforward. This restriction, however, makes the task of programming more difficult. Conversely, variable-sized messages require a more complex system-level implementation, but the programming task becomes simpler. This is a common kind of tradeoff seen throughout operating system design.
Naming: Processes that want to communicate must have a way to refer to each other. They can use either direct or indirect communication. Under direct communication, each process that wants to communicate must explicitly name the recipient or sender of the communication. In this scheme, the send.0 and receive() primitives are defined as:
• send(P, message)—Send a message to process P.
• receive (Q, message)—Receive a message from process Q.
A communication link in this scheme has the following properties:
• A link is established automatically between every pair of processes that want to communicate. The processes need to know only each other's identity to communicate.
• A link is associated with exactly two processes.
• Between each pair of processes, there exists exactly one link.
The disadvantage in both of these schemes (symmetric and asymmetric) is the limited modularity of the resulting process definitions. Changing the identifier of a process may necessitate examining all other process definitions.
Synchronization
Communication between processes takes place through calls to send() and receive () primitives. There are different design options for implementing each primitive.
Message passing may be either **blocking** or **nonblocking**— also known as **synchronous** and **asynchronous.**
• **Blocking send-** The sending process is blocked until the message is received by the receiving process or by the mailbox.
• **Nonblocking send-** The sending process sends the message and resumes operation.
• **Blocking receive-** The receiver blocks until a message is available.
• **Nonblocking receive-** The receiver retrieves either a valid message or a null.
Buffering
Whether communication is direct or indirect, messages exchanged by communicating processes reside in a temporary queue. Basically, such queues can be implemented in three ways:
• **Zero capacity-** The queue has a maximum length of zero; thus, the link cannot have any messages waiting in it. In this case, the sender must block until the recipient receives the message.
• **Bounded capacity-** The queue has finite length *n;* thus, at most *n* messages can reside in it. If the queue is not full when a new message is sent, the message is placed in the queue (either the message is copied or a pointer to the message is kept), and the sender can continue execution without waiting. The links capacity is finite, however. If the link is full, the sender must block until space is available in the queue.
• **Unbounded capacity-** The queues length is potentially infinite; thus, any number of messages can wait in it. The sender never blocks.

4.0 OPERATING SYSTEM PROCESSES

A process is a program in execution. The execution of a process must progress in a sequential fashion. Definition of process is following.
A process is defined as an entity which represents the basic unit of work to be implemented in the system.
Components of a process are following.

S.N.	Component & Description
1	Object Program: Code to be executed.
2	Data: Data to be used for executing the program.
3	Resources: While executing the program, it may require some resources.
4	Status: Verifies the status of the process execution. A process can run to completion only when all requested resources have been allocated to the process.

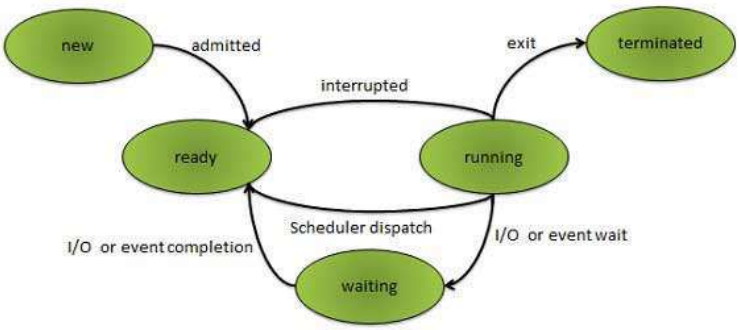
Difference between a Program and a Process
A program by itself is not a process. It is a static entity made up of program statement while process is a dynamic entity. Program contains the instructions to be executed by processor.

A program takes a space at single place in main memory and continues to stay there. A program does not perform any action by itself.

Process States : As a process executes, it changes state. The state of a process is defined as the current activity of the process.

S.N.	State & Description
1	New The process is being created.
2	Ready The process is waiting to be assigned to a processor. Ready processes are waiting to have the processor allocated to them by the operating system so that they can run.
3	Running Process instructions are being executed (i.e. The process that is currently being executed).
4	Waiting The process is waiting for some event to occur (such as the completion of an I/O operation).
5	Terminated The process has finished execution.

The figure below depicts the relationships between the process states:



Process Control Block, PCB

Each process is represented in the operating system by a process control block (PCB) also called a task control block. PCB is the data structure used by the operating system. Operating system groups all information that needs about particular process.

PCB contains many pieces of information associated with a specific process which is described below.

S/N.	Information & Description
1	Pointer Pointer points to another process control block. Pointer is used for maintaining the scheduling list.
2	Process State Process state may be new, ready, running, waiting and so on.
3	Program Counter Program Counter indicates the address of the next instruction to be executed for this process.
4	CPU registers CPU registers include general purpose register, stack pointers, index registers and accumulators etc. number of register and type of register totally depends upon the computer architecture.
5	Memory management information This information may include the value of base and limit registers, the page tables, or the segment tables depending on the memory system used by the operating system. This information is useful for deallocating the memory when the process terminates.
6	Accounting information This information includes the amount of CPU and real time used, time limits, job or process numbers, account numbers etc.

5.0 Process Scheduling Algorithms

We'll discuss four major scheduling algorithms here which are following

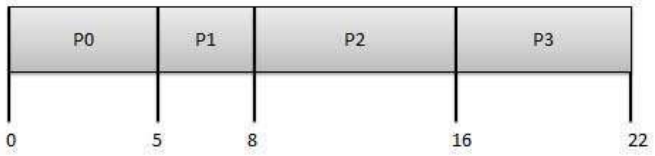
i. First Come First Serve (FCFS) Scheduling
ii. Shortest-Job-First (SJF non-preemptive) Scheduling
iii. Shortest-Job-First (SJF preemptive) Scheduling
iv. Priority Scheduling
v. Round Robin (RR) Scheduling

First Come First Serve (FCFS)

- Jobs are executed on first come, first serve basis.
- Easy to understand and implement.
- Poor in performance as average wait time is high.

➢

Process	Arrival Time	Execute Time	Service Time
P0	0	5	0
P1	1	3	5
P2	2	8	8
P3	3	6	16



Wait time of each process is following

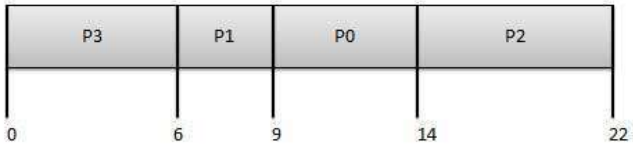
Process	Wait Time : Service Time - Arrival Time
P0	0 - 0 = 0
P1	5 - 1 = 4
P2	8 - 2 = 6
P3	16 - 3 = 13

Average Wait Time: (0+4+6+13) / 4 = 5.55

Priority Based Scheduling

- Each process is assigned a priority. Process with highest priority is to be executed first and so on.
- Processes with same priority are executed on first come first serve basis.
- Priority can be decided based on memory requirements, time requirements or any other resource requirement.

Process	Arrival Time	Execute Time	Priority	Service Time
P0	0	5	1	0
P1	1	3	2	3
P2	2	8	1	8
P3	3	6	3	16



Wait time of each process is following

Process	**Wait Time : Service Time - Arrival Time**
P0	0 - 0 = 0
P1	3 - 1 = 2
P2	8 - 2 = 6
P3	16 - 3 = 13

Average Wait Time: (0+2+6+13) / 4 = 5.25

Short Job First (SJF) has two schemes:

i. Non preemptive: once CPU given to a process it cannot be preempted until completes its CPU burst time.
ii. Preemptive: if a new process arrives with CPU burst length less than the remaining time of the current executing process, preempt. This scheme is also known as Short-Remaining-Time-First (SRTF).

SJF is optimal – given minimum average waiting time for a give set of processes

Example: SJF (non-preemptive/preemptive)

Process	**Arrival time**	**Burst time**
P1	0.0	7
P2	2.0	4
P3	4.0	1
P4	5.0	4

SJF (non-preemptive)

P1	**P3**	**P2**	**P4**
0	**7**	**8**	**12**	**16**

Average Wait Time: (0+6+3+7) / 4 = 4

SJF (preemptive)

P1	**P2**	**P3**	**P2**	**P4**	**P1**	
0	**2**	**4**	**5**	**7**	**11**	**16**

Average Wait Time: (9+1+0+2) / 4 = 3

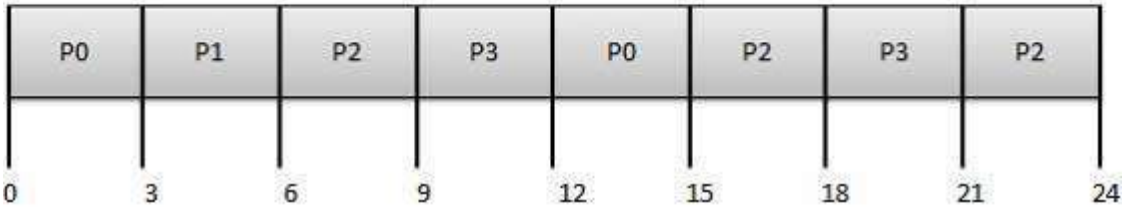
Round Robin Scheduling

- Each process is provided a fix time to execute called quantum.

- Once a process is executed for given time period. Process is preempted and other process executes for given time period.
- Context switching is used to save states of preempted processes.

Process	Arrival Time	Execute Time
P0	0	5
P1	1	3
P2	2	8
P3	3	6

Quantum = 3



Average Wait Time: (9+2+10+12) / 4 = 8.25

6.0 INTERRUPT AND MASKING TRAPS

5.1 An interrupt vector is the memory location of an interrupt handler, which prioritizes interrupts and saves them in a queue if more than one interrupt is waiting to be handled.

5.2 An interrupt is a signal from a device attached to a computer, or from a program within the computer, that tells the OS (operating system) to stop and decide what to do next. When an interrupt is generated, the OS saves its execution state by means of a context switch, a procedure that a computer processor follows to change from one task to another while ensuring that the tasks do not conflict. Once the OS has saved the execution state, it starts to execute the interrupt handler at the interrupt vector.

5.3 Interrupt masking allows you to disable the detection of an interrupt-line assertion, which causes the OS to ignore interrupt signal. The signal can be ignored at the microprocessor level or at other levels in the hardware architecture.

In some cases, each interrupt source in the system can be masked individually. In other cases, masking an interrupt in a microprocessor register can mask a group of interrupts. An example of this is shared interrupts.

When an interrupt occurs, the microprocessor must globally disable interrupts at the microprocessor level to avoid being interrupted while gathering and saving interrupt state information. Because disabling interrupts globally blocks all other interrupts, mask global interrupts for as short a time as possible in your OAL ISR code. When you determine what has specifically interrupted, you can mask just that interrupt.

5.4 A trap is a type of synchronous interrupt typically caused by an exceptional condition (e.g. division by zero or invalid memory access) in a user process. A trap usually results in a switch to kernel mode, wherein the operating system performs some action before returning control to the originating process. In some usages, the term trap refers specifically to an interrupt intended to initiate a context switch to a monitor program or debugger.

5.5 Traps and interrupts are events that break down the normal execution of the program.

Trap: A trap is and abnormal condition detected by the CPU, which indicates an unknown I/O device is accessed, etc.

Interrupt: An interrupt in an interruption in the normal execution of the program. When the CPU is interrupt, then it stops its current activities like execution of the program. And transfer the control to interrupting device to check the interrupt. The CPU responds interrupt. By saving the current value of the program counter and resetting the program to a new address. The new address is the starting address where procedure for handling interrupt is located. Similarly, other state information is also saved when an interrupt occurs. In many computer systems this information is stored in a special program status word register. After saving the necessary information, the interrupt service routine is executed on completion; the CPU resumes the interrupted program. Thus the application program does not have to contain any special code to accommodate the interrupts. The CPU and the operating system are responsible for suspending the program and then resuming it at the end of interrupt processing.

Difference:

1. A trap will occur at exactly the same point of the program execution, each time a program runs.
2. An interrupt is dependent on the relative timing between the interrupting device and the CPU.

An interrupt is signal sent to the CPU by an external hardware device such as I/O device. The software can also send interrupt signal to the CPU. Hardware may trigger an interrupt at any time sending a signal to the CPU through system bus. It also called a monitor call or supervisor call.

Interrupt are the important part of computer architecture. Each machine has its own interrupt mechanism, but most of the functions are common. The interrupt must transfer control to the appropriate interrupt service routine. Similarly, interrupts must be handled quickly. In a computer system only a predefined interrupts can be occurred, so the array of pointer is used to store the address of interrupt routines. This array of pointer is called the interrupt vector. Each trap and interrupt is associated with an index into that vector. The index values are the unique device number that provides the address of the interrupt service routine for interrupting device. The early computer systems stored the interrupt address in a fixed location or in a location indexed by the device number but the modern computer use the stack for this purpose.

Classes of interrupts:

There are many classes of interrupts, but the most common classes of interrupts are

- Program check interrupts
- Supervisor call interrupts
- Timer interrupts
- I/O interrupts
- External interrupts
- Machine check interrupts

Program check interrupts:

These are caused by some problems that may occur during program execution such as division by zero, arithmetic overflow or underflow, data read in correct format, attempt to reference a memory location beyond the limits of real memory, attempt to reference a protected resource etc. many systems allow users to specify their own routines to be executed when a program check interrupt occurs.

Supervisor call interrupts:

These are initiated by a running process that executes the supervisor call instruction. This type of interrupt is a user generated request for a particular system service such as for I/O operation etc.

Timer interrupts:

These are generated by timer within the CPU. Timer interrupt allows the operating system to perform certain function on regular intervals of time.

I/O interrupts:

These are generated by I/O controller. The i/o interrupts signal to the cpu that the status of device has changed. I/O interrupts are caused due to three resigns.

1. Input/output operation completes.
2. Input/output error occurs.
3. Input/output device is made ready.

External interrupts:

These are caused by pressing of the console's interrupt key by the operation or the receipt of a signal from another processor on a multiprocessor system.

Machine check interrupts:

These are caused by hardware failure such as memory parity error.

6.0 The Kernel

The kernel is the essential center of a computer operating system, the core that provides basic services for all other parts of the operating system. A synonym is nucleus. A kernel can be contrasted with a shell, the outermost part of an operating system that interacts with user commands. Kernel and shell are terms used more frequently in Unix operating systems than in IBM mainframe or Microsoft Windows systems.

Typically, a kernel (or any comparable center of an operating system) includes an interrupt handler that handles all requests or completed I/O operations that compete for the kernel's services, a scheduler that determines which programs share the kernel's processing time in what order, and a supervisor that actually gives use of the computer to each process when it is scheduled. A kernel may also include a manager of the operating system's address spaces in memory or storage, sharing these among all components and other users of the kernel's services. A kernel's services are requested by other parts of the operating system or by application programs through a specified set of program interfaces sometimes known as system calls.

Because the code that makes up the kernel is needed continuously, it is usually loaded into computer storage in an area that is protected so that it will not be overlaid with other less frequently used parts of the operating system.

Types of Kernels

Kernels may be classified mainly in two categories

1. Monolithic (ii) Micro Kernel (iii) Hybrid

1. Monolithic Kernels: Earlier in this type of kernel architecture, all the basic system services like process and memory management, interrupt handling etc were packaged into a single module in kernel space. This type of architecture led to some serious drawbacks like 1) Size of kernel, which was huge. 2) Poor maintainability, which

means bug fixing or addition of new features resulted in recompilation of the whole kernel which could consume hours.

In a modern day approach to monolithic architecture, the kernel consists of different modules which can be dynamically loaded and un-loaded. This modular approach allows easy extension of OS's capabilities. With this approach, maintainability of kernel became very easy as only the concerned module needs to be loaded and unloaded every time there is a change or bug fix in a particular module. So, there is no need to bring down and recompile the whole kernel for a smallest bit of change. Also, stripping of kernel for various platforms (say for embedded devices etc) became very easy as we can easily unload the module that we do not want.

2 Microkernels: This architecture majorly caters to the problem of ever growing size of kernel code which we could not control in the monolithic approach. This architecture allows some basic services like device driver management, protocol stack, file system etc to run in user space. This reduces the kernel code size and also increases the security and stability of OS as we have the bare minimum code running in kernel. So, if suppose a basic service like network service crashes due to buffer overflow, then only the networking service's memory would be corrupted, leaving the rest of the system still functional.

In this architecture, all the basic OS services which are made part of user space are made to run as servers which are used by other programs in the system through inter process communication (IPC). eg: we have servers for device drivers, network protocol stacks, file systems, graphics, etc. Microkernel servers are essentially daemon programs like any others, except that the kernel grants some of them privileges to interact with parts of physical memory that are otherwise off limits to most programs. This allows some servers, particularly device drivers, to interact directly with hardware. These servers are started at the system start-up.

So, what the bare minimum that microKernel architecture recommends in kernel space?

- Managing memory protection
- Process scheduling
- Inter Process communication (IPC)

Apart from the above, all other basic services can be made part of user space and can be run in the form of servers.

While monolithic kernels execute all of their code in the same address space (kernel space) microkernels try to run most of their services in user space, aiming to improve maintainability and modularity of the codebase. Most kernels do not fit exactly into one of these categories, but are rather found in between these two designs. These are called hybrid kernels

3. Hybrid (or Modular) kernels

Hybrid kernels are used in most commercial operating systems such as Microsoft Windows NT 3.1, NT 3.5, NT 3.51, NT 4.0, 2000, XP, Vista, 7, 8, 8.1 and 10. Apple Inc's own Mac OS X uses a hybrid kernel called XNU which is based upon code from Carnegie Mellon's Mach kernel and FreeBSD's monolithic kernel. They are similar to micro kernels, except they include some additional code in kernel-space to increase performance. These kernels represent a compromise that was implemented by some developers before it was demonstrated that pure micro kernels can provide high performance. These types of kernels are extensions of micro kernels with some properties of monolithic kernels. Unlike monolithic kernels, these types of kernels are unable to load modules at runtime on their own. Hybrid kernels are micro kernels that have some "non-essential" code in kernel-space in order for the code to run more quickly than it would were it to be in user-space. Hybrid kernels are a compromise between the monolithic and microkernel designs. This implies running some services (such as the network stack or the file system) in kernel space to reduce the performance overhead of a traditional microkernel, but still running kernel code (such as device drivers) as servers in user space.

A few advantages to the modular (or) Hybrid kernel are:

- Faster development time for drivers that can operate from within modules.
- On demand capability versus spending time recompiling a whole kernel for things like new drivers or subsystems.
- Faster integration of third party.

Some of the disadvantages of the modular approach are:

- With more interfaces to pass through, the possibility of increased bugs exists (which implies more security holes).
- Maintaining modules can be confusing for some administrators when dealing with problems like symbol differences.

7.0 OPERATION SYSTEM COMMANDS

The command interpreter for DOS runs when no application programs are running. When an application exits, if the transient portion of the command interpreter in memory was overwritten, DOS will reload it from disk. Some commands are internal — built into COMMAND.COM; others are external commands stored on disk. When the user types a line of text at the operating system command prompt,

External commands were too large to keep in the command processor, or were less frequently used. Such utility programs would be stored on disk and loaded just like regular application programs but were distributed with the operating system.
The command interpreter preserves the case of whatever parameters are passed to commands, but the command names themselves and filenames are case-insensitive.
Many commands are the same across many DOS systems, but some differ in command syntax or name.
Windows Commands

Command	Description
attrib	Displays or changes file attributes
chdir (cd)	Displays the name of the current directory or changes the current directory
chkdsk	Displays a disk status report and corrects errors on the disk
cls	Clears the screen
cmd	Starts a new instance of the Windows NT command interpreter
copy	Copies one or more files to another location
date	Displays the date or allows you to change the date
del (erase)	Deletes specified files
dir	Displays a list of a directory's files and subdirectories
diskcopy	Copies a floppy disk
format	Formats a disk to accept Windows NT files
mkdir (md)	Creates a directory or subdirectory
move	Moves one or more files to a specified directory
rename (ren)	Changes the name of a file or files
rmdir (rd)	Deletes (removes) a directory
time	Displays the system time or sets the computer's internal clock
tree	Displays the directory structure of a path or disk
ver	Displays the Windows NT version number
vol	Displays the disk volume label and serial number
xcopy	Copies files and directories, including subdirectories

Revision Questions

Define Operating system. Hence list the different classification of operating system.

With the aid of well-labeled diagram, show the relationships between the different process states.

State the reasons for allowing process cooperation.

Given the data in the table below, determine which of the following process scheduling algorithms with give least average waiting time: (i) FCFS (ii) SRTF

Process	Arrival Time	Burst Time
P1	0.0	7
P2	2.0	4
P3	4.0	1
P4	5.0	4

5. (i) Define Interrupt (ii) Why the need for interrupt masking?
(iii) List any six common classes of Interrupt known to you. (iv) List the different categories of OS Kernel
(v) Interpret the following OS commands: DIR, CD, MKDIR, CLS